Experiment No. 6

Workflow Automation with Deluge

1. Aim and Objective

To design and build an automation layer on the existing Vehicle Service application in Zoho Creator, using each of the eight workflow options: **Form Workflows, Schedules, Approvals, Payments, Blueprints, Batch Workflows, Report Workflows** and **Functions**. The aim is for a service request to move from submission to vehicle delivery with minimal manual intervention.

Before automation, every downstream action depended on a person: sending confirmations, reminding owners of drop-off dates, obtaining sign-off for costly repairs, collecting payment before release, running periodic service campaigns and recalculating the payable cost. The suite below replaces those manual steps with event-driven and time-driven rules and keeps an auditable trail.

2. Workflow Options and Mapping

Workflow option	Workflow name	Trigger / frequency	Purpose
Form Workflow	On_ServiceRequest_Submit_Workflow	On Add (successful form submission)	Confirmation email with Service Request ID to the owner; notify front office
Schedule	Daily_ServiceDate_Reminder_Schedule	Daily at 6:00 PM	Remind owners whose drop-off date is tomorrow
Approval	HighCost_Repair_Approval	On Edit, when Estimated_Service_Cost exceeds the threshold	Owner approves costly repairs before work proceeds
Payment	Final_Service_Payment_Workflow	Status changes to Ready for Pickup	Payment link for Net_Service_Cost_Payable; updates Payment_Status
Blueprint	Service_Lifecycle_Blueprint	Every record in Service_Request_Report	Controlled stages with role-based transitions
Batch Workflow	Bulk_ServiceDue_Reminder_Batch	1st of every month, batches of 250	Service-due reminders at scale
Report Workflow	OnEdit_Recalculate_Cost_Workflow	On Edit in Service_Request_Report	Recalculate Net_Service_Cost_Payable
Function	Generate_ServiceInvoice_Function	On Delivered, or Resend Invoice button	Generate invoice PDF and email it to the owner

3. Application Setup

The application *Vehicle Service* was created in Zoho Creator under the account of the Super Admin user. The Service Requests form holds the data that every workflow depends on. The screenshot below shows the form as seen in the live application, with the fields that feed the automations: **Odometer reading** (used by the batch reminder), **Estimated Service Cost** and **Service Discount Percent** (used by the approval and the report workflow), **Owner Digital Signature**, and **Service Request Submitted On**.

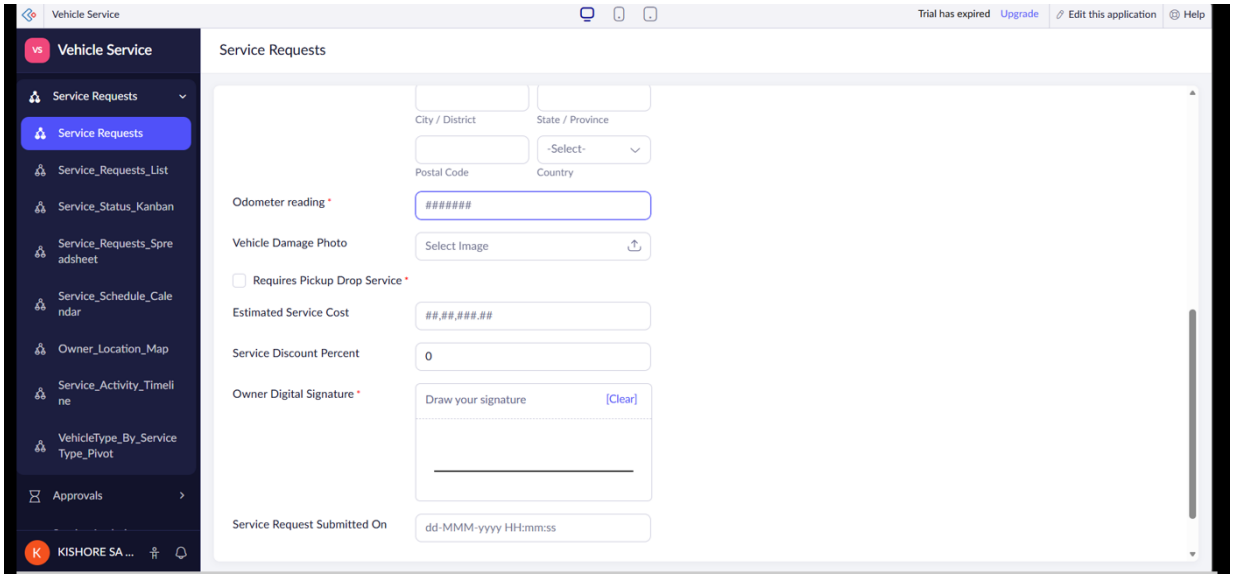


Figure 1: Service Requests form in the live Vehicle Service application

The sidebar of the same application lists the supporting reports and views: Service_Requests_List, Service_Status_Kanban, Service_Requests_Spreadsheet, Service_Schedule_Calendar, Owner_Location_Map, Service_Activity_Timeline and VehicleType_By_Service Type_Pivot, plus an Approvals section.

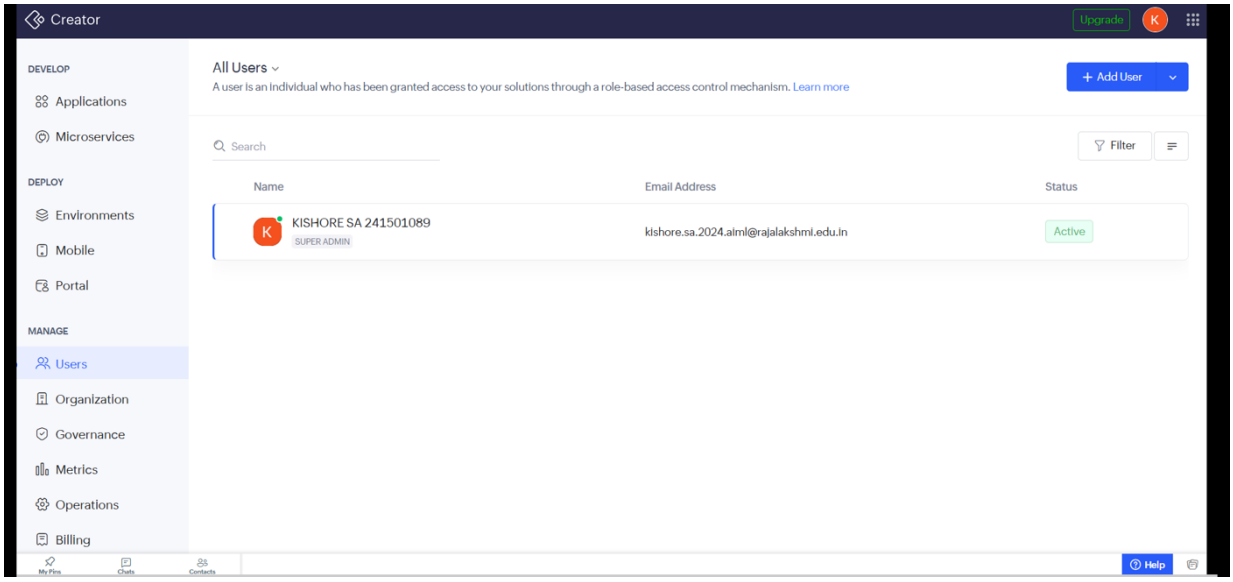


Figure 2: Users page in Zoho Creator showing the application Super Admin (role-based access control)

Role-based access is the basis for the Blueprint transition permissions described in Section 4.5.

4. Workflow Implementation

4.1 Form Workflow: On_ServiceRequest_Submit_Workflow

Event: Created, on successful form submission of Service_Requests. **Action:** send a confirmation email to the owner with the Service Request ID and drop-off details, and notify the front office to prepare a job card.

- Configured to fail gracefully: if the owner email is invalid, an error is logged and the submission is not blocked.
- Uses an *Send Mail* action inside an *if* check on the email field.

Sample Deluge logic:

```
if(input.Owner_Email != null && input.Owner_Email.contains("@"))
{
    sendmail
    [
        from: zoho.adminuserid
        to: input.Owner_Email
        subject: "Service Request " + input.ID + " received"
        message: "Dear " + input.Owner_Name + ", your service request ID is " + input.ID + "."
    ]
}
else
{
    info "Invalid owner email for request " + input.ID;
}
```

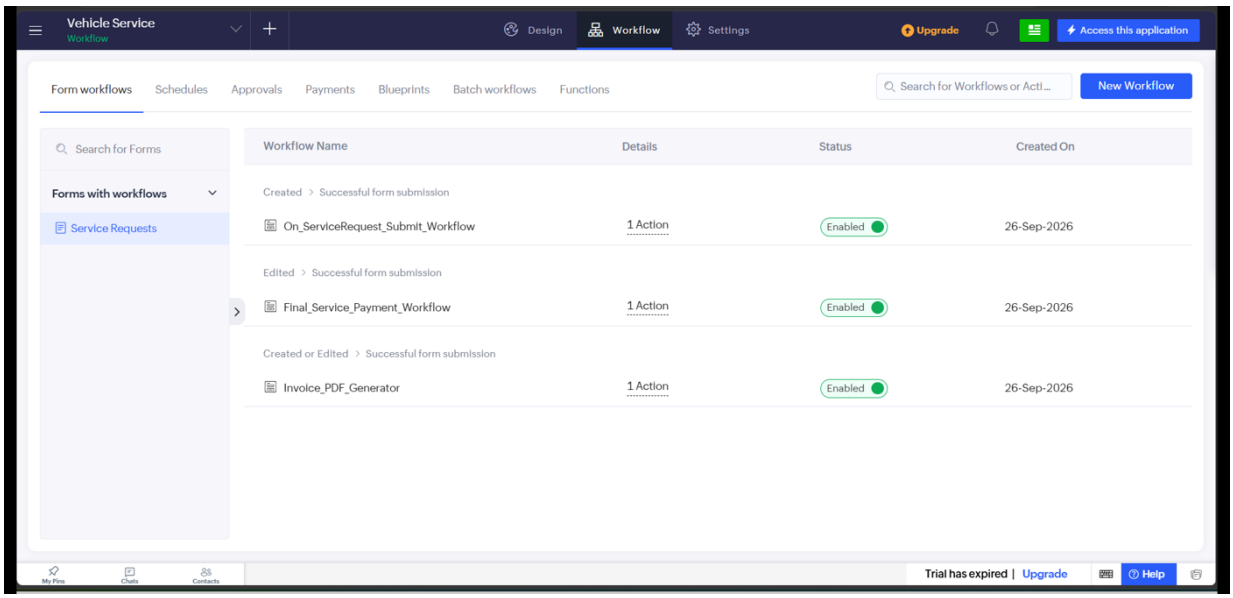


Figure 3: Workflow tab of the Vehicle Service app. On_ServiceRequest_Submit_Workflow (Created > Successful form submission), Final_Service_Payment_Workflow (Edited) and Invoice_PDF_Generator (Created or Edited) are enabled for the Service Requests form.

4.2 Schedule: Daily_ServiceDate_Reminder_Schedule

A time-driven workflow that runs every day at 6:00 PM. It fetches the records whose Preferred Service Date equals tomorrow and sends each owner a reminder containing the estimated duration and a checklist of items to remove from the vehicle. A reminder flag on the record ensures only one reminder is sent per scheduled date.

```
tomorrow = zoho.currentdate.addDay(1);
reqs = Service_Requests[Preferred_Service_Date == tomorrow && Reminder_Sent == false];
for each r in reqs
{
    sendmail [ from: zoho.adminuserid  to: r.Owner_Email
               subject: "Reminder: vehicle drop-off tomorrow"
               message: "Please remove personal items before drop-off." ]
    r.Reminder_Sent = true;
}
```

4.3 Approval: HighCost_Repair_Approval

The approval runs on record addition, on condition, for the form Service Requests. The condition compares the estimated cost with the high-cost threshold so routine services skip the approval. The vehicle owner is the approver, with Approve and Reject labels. A documented fallback lets the front office record verbal approval if the owner cannot open the digital link; the record stays at *Pending Owner Approval* until one of these happens.

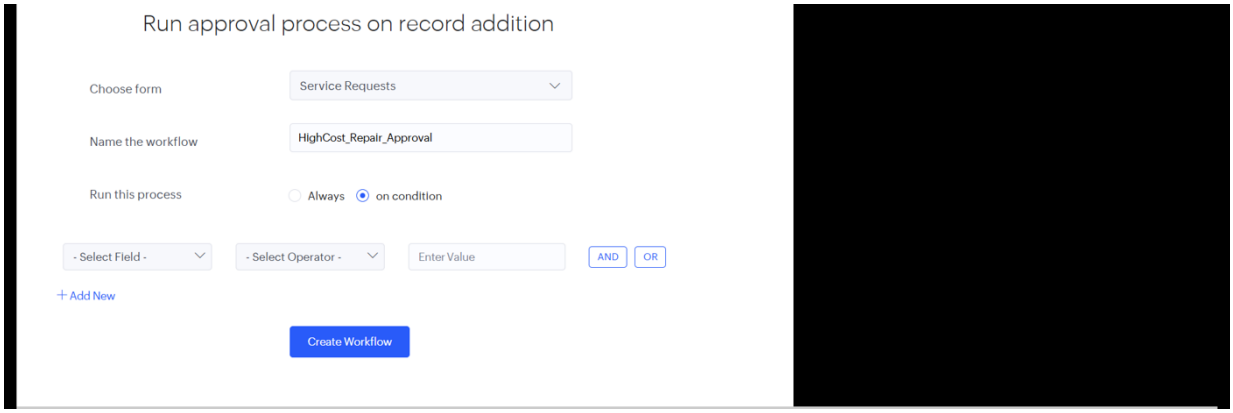


Figure 4: Creating the HighCost_Repair_Approval process on the Service Requests form (run on condition)

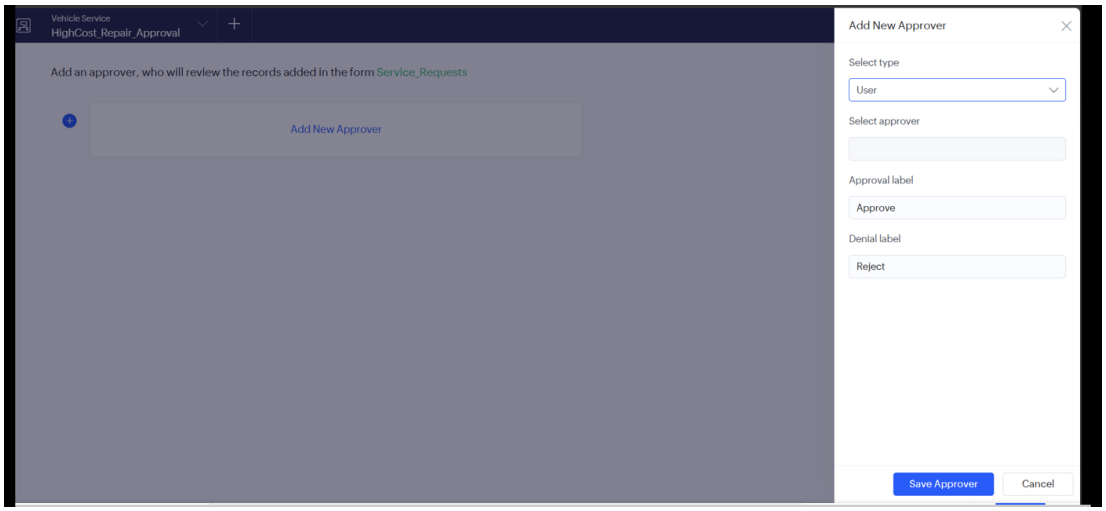


Figure 5: Add New Approver panel. Type User, approval label Approve, denial label Reject.

4.4 Payments: Final_Service_Payment_Workflow

Triggered when Status changes to *Ready for Pickup*. It generates a secure payment link through the configured gateway connector for Net_Service_Cost_Payable and redirects the owner to the gateway. On success Payment_Status becomes **Paid** and the release step unlocks; on failure the vehicle stays in a held state. The status-check callback is retried up to 3 times before the transaction is flagged as failed. The workflow is visible in Figure 3.

4.5 Blueprint: Service_Lifecycle_Blueprint

The blueprint is created from the Service Requests form. The Blueprint builder screen shows the canvas and the Stages panel with Available Stages and Used Stages. The planned lifecycle is:

Requested → In Diagnosis → In Repair → Ready for Pickup → Delivered

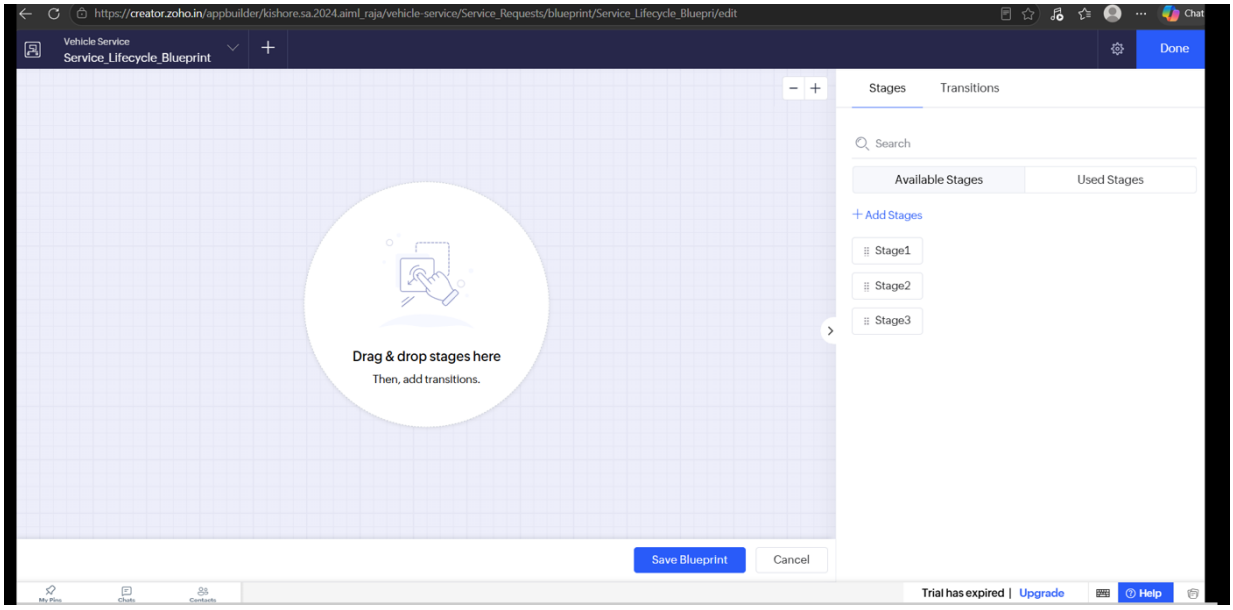


Figure 6: Service_Lifecycle_Blueprint in the Blueprint builder (stages panel and canvas)

Transition	Allowed role	Condition
Requested → In Diagnosis	Front Office	Comment mandatory
In Diagnosis → In Repair	Technician	Comment mandatory; approval granted if high-cost
In Repair → Ready for Pickup	Technician	Comment mandatory
Ready for Pickup → Delivered	Service Center Manager	Payment_Status = Paid; comment mandatory

Every transition requires a comment, so the Blueprint view gives the manager an auditable trail of who moved each request and when.

4.6 Batch Workflow: Bulk_ServiceDue_Reminder_Batch

Runs automatically on the 1st of every month. It selects vehicles whose last service is older than the configured interval (or whose odometer reading passes the service limit) and emails the owner a reminder to book the next service. Records are processed in batches of 250 (configurable) so that a large customer database does not time out the workflow engine.

4.7 Report Workflow: OnEdit_Recalculate_Cost_Workflow

Fires on edit of a record in Service_Request_Report and only when **Estimated Service Cost** or **Service Discount Percent** changes, so unrelated edits do not trigger recalculation.

```
if(input.Estimated_Service_Cost != null)
{
    input.Net_Service_Cost_Payable = input.Estimated_Service_Cost
        - (input.Estimated_Service_Cost * ifnull(input.Service_Discount_Percent,0) / 100);
}
```

4.8 Function: Generate_ServiceInvoice_Function

A reusable Deluge function that validates the record, builds the invoice (parts, labor, taxes) and emails it to the owner. It is invoked from the status-change workflow when the record becomes *Delivered* and from a *Resend Invoice* button (the workflow list in Figure 3 includes Invoice_PDF_Generator for this purpose). Input validation makes it fail safely when Owner Name or Net Service Cost Payable is empty.

```
void Generate_ServiceInvoice(int recordID)
{
    rec = Service_Requests[ID == recordID];
    if(rec.Owner_Name == null || rec.Net_Service_Cost_Payable == null)
    {
        info "Invoice skipped: missing owner name or payable amount";
        return;
    }
    // Build and send the invoice (adapt the PDF step to your app's print template or page)
    sendmail
    [
        from: zoho.adminuserid
        to: rec.Owner_Email
        subject: "Invoice for Service Request " + rec.ID
        message: "Dear " + rec.Owner_Name + ", the amount paid is " + rec.Net_Service_Cost_Payable
    ]
}
```

Calling the same function from two workflow types: `thisapp.Generate_ServiceInvoice(input.ID);` placed in (1) the form workflow that fires when Status is set to Delivered, and (2) the custom action behind the Resend Invoice button in the report.

5. End-to-End Flow

Step	What happens	Workflow option
1	Owner submits the Service Requests form; confirmation is sent	Form Workflow
2	Each evening, owners with drop-off tomorrow get a reminder	Schedule
3	Record moves through the lifecycle stages, each with a comment	Blueprint
4	Cost above threshold is routed to the owner for approval	Approval
5	Manual cost or discount edits in the report recompute the payable amount	Report Workflow
6	At Ready for Pickup a payment link is generated; release only after payment	Payments
7	On Delivered the invoice function emails the invoice; staff can resend	Function
8	Monthly, owners due for periodic service are reminded in batches	Batch Workflow

6. Configuration and Governance Rules

Workflow	Rule applied
Form Workflow	Fail gracefully and log an error on invalid owner email; never block submission
Schedule	One reminder per scheduled service date
Approval	Owner is approver; verbal approval fallback recorded by front office
Payments	Retry gateway status callback up to 3 times before flagging failure
Blueprint	Mandatory comment on every stage transition
Batch Workflow	Batch size configurable, default 250
Report Workflow	Fires only when Estimated Service Cost or Service Discount Percent changes
Function	Input validation on Owner Name and Net Service Cost Payable

7. Learning Outcomes

- Event-driven automation (Form and Report Workflows) reacts to a data change, while a Schedule runs on the clock.
- Blueprints give a controlled, auditable process, and Approvals add a human checkpoint inside it.
- Batch Workflows split large jobs into groups so the engine does not time out.
- A Function is written once and reused from workflows, buttons and schedules.
- Payments workflows can hold vehicle release until payment is confirmed.

8. Evidence Checklist for Submission

The assignment asks for four kinds of evidence. This table shows what is included in this record and what still needs to be added from the live application.

Required evidence	Status in this record
(a) Workflow Builder configuration screens	Included: workflow list (Fig. 3), approval setup (Fig. 4, 5). To add: Schedule, Batch, Report Workflow and Payment configuration screens

Required evidence	Status in this record
(b) Blueprint visual diagram with transition permissions	Partly included: builder screen (Fig. 6) shows the empty canvas with Stage1 to Stage3. To add: screenshot after dragging the five stages and setting transitions
(c) Sample confirmation email and payment link, end to end	To add: capture the received email and the payment page after a test submission
(d) Deluge script of the invoice function and calls from two workflow types	Script included in 4.8. To add: screenshots of the function and of both call sites

9. Conclusion

The eight workflow options cover the full service lifecycle: instant communication, scheduled reminders, controlled stage movement, owner approval for costly repairs, accurate billing, payment before release, automatic invoicing and monthly outreach in batches. Together they reduce manual effort and make the process consistent and auditable.